

# Super Mario Game — Architecture, System Design & Spec Questionnaire

## 1. Game Engine Recommendation: SFML 3.0.2

[!IMPORTANT] **Recommendation: SFML 3.0.2** — already configured and compiled in our workspace.

| Criteria          | SFML 3.0.2                                                | SDL2/SDL3               | Raylib                  |
|-------------------|-----------------------------------------------------------|-------------------------|-------------------------|
| Language          | Native C++ (OOP)                                          | C (procedural)          | C (procedural)          |
| OOP Fit           | ★★★★★ Natural                                             | ★★★★☆ Requires wrappers | ★★★★☆ Requires wrappers |
| Prior Experience  | ✅ Team used SFML in CS163 project                         | ❌ None                  | ❌ None                  |
| Modules           | Graphics, Window, Audio, Network, System                  | Render, Audio, Input    | All-in-one              |
| ImGui Integration | ✅ Official ImGui-SFML binding                             | Requires manual setup   | Requires manual setup   |
| 2D Platformer Fit | Excellent sprite batching, texture atlases, views/cameras | Excellent but verbose   | Good but less control   |
| Build System      | CMake FetchContent (already working)                      | CMake (more setup)      | CMake (simple)          |

**Why SFML?** 1. **OOP-native:** SFML's classes ( `sf::Sprite` , `sf::Texture` , `sf::Sound` ) map naturally to our C++ OOP architecture — no C-to-C++ wrapper boilerplate. 2. **Team familiarity:** The team has shipped a full SFML project (DataStructureVisualizer) with this exact version. 3. **Already compiled:** Our `build/` already produces a working SFML 3.0.2 + ImGui binary in <5 minutes. 4. **ImGui-SFML:** Gives us free developer tools (entity inspector, level editor UI) with zero additional setup.

## 2. System Architecture Overview

---

We use a **Layered Architecture** with clear dependency boundaries. Higher layers depend on lower layers, never the reverse.

```

graph TD
    subgraph "Application Layer"
        A["main.cpp<br/>Entry Point"]
    end

    subgraph "Core Layer"
        B["Game<br/>(Singleton)"]
        C["GameStateManager<br/>(State Pattern)"]
        D["InputManager"]
        E["ResourceManager<br/>(Singleton)"]
        F["SoundManager<br/>(Singleton)"]
    end

    subgraph "Game States"
        G["MenuState"]
        H["CharSelectState"]
        I["PlayingState"]
        J["PauseState"]
        K["GameOverState"]
        L["VictoryState"]
    end

    subgraph "World Layer"
        M["World"]
        N["Level / TileMap"]
        O["Camera"]
        P["HUD"]
    end

    subgraph "Entity Layer"
        Q["Entity (Base)"]
        R["Character (Base)"]
        S["Mario / Luigi"]
        T["Enemy (Base)"]
        U["Goomba / KoopaTroopa"]
        V["Item (Base)"]
        W["Mushroom / FireFlower / Coin / Star"]
        X["Block (Base)"]
        Y["BrickBlock / QuestionBlock / Pipe"]
    end

    subgraph "Physics Layer"
        Z["PhysicsEngine"]
        AA["AABB Collider"]
        AB["CollisionResolver"]
    end

    subgraph "Infrastructure Layer"
        AC["Renderer (SFML Wrapper)"]
    end

```

```

    AD["AnimationManager"]
    AE["Serializer (Save/Load)"]
    AF["EntityFactory<br/>(Factory Pattern)"]
    AG["EventBus<br/>(Observer Pattern)"]
end

A → B
B → C
B → D
B → E
B → F
C → G & H & I & J & K & L
I → M
M → N & O & P
M → Q
Q → R & V & X
R → S & T
T → U
V → W
X → Y
M → Z
Z → AA & AB
B → AC & AD & AE & AF & AG

```

### 3. Design Patterns Mapping (5 Required + 5 Bonus = 10+)

Each pattern is mapped to a concrete class and its rubric justification.

| # | Pattern   | Class(es)                                                                                                                                                                                   | Purpose                                                                                                                                                                                                                                           | Rubric Points |
|---|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1 | Factory   | <code>EntityFactory</code>                                                                                                                                                                  | Spawns enemies, items, blocks from tilemap data.<br><code>createEntity(EntityType, position)</code> returns <code>std::unique_ptr&lt;Entity&gt;</code> . Lakitu uses Factory to spawn Spinies at runtime.                                         | 5 pts         |
| 2 | Singleton | <code>Game</code> , <code>ResourceManager</code> , <code>SoundManager</code> , <code>AchievementManager</code>                                                                              | Single instances of core managers. Lazy-init with <code>getInstance()</code> .                                                                                                                                                                    | 5 pts         |
| 3 | State     | <code>GameStateManager</code> / <code>IGameState</code> , <code>IPlayerState</code> (5 concrete states: Small, Super, Fire, Cape, Mini), <code>FallingPlatform</code> , <code>Thwomp</code> | Game flow (9 states). Player power forms (5 permanent states). Entity lifecycles. <b>Note:</b> Star and Mega are not states — they are Decorators (see #7).                                                                                       | 5 pts         |
| 4 | Observer  | <code>EventBus</code>                                                                                                                                                                       | Decoupled event system. 15+ event types. HUD, SoundManager, ComboTracker, AchievementTracker, StatisticsTracker subscribe independently.                                                                                                          | 5 pts         |
| 5 | Strategy  | <code>IMovementStrategy</code>                                                                                                                                                              | Enemy AI behaviors: 7+ strategies (Patrol, Chase, Fly, TimerEmergence, Linear, HammerThrow, TetheredChase, ProximityTrigger). Swappable at runtime. Enemy <code>update()</code> delegates entirely to strategy — no Template Method hooks needed. | 5 pts         |
| 6 | Command   | <code>ICommand</code> + <code>InputManager</code>                                                                                                                                           | 8+ commands. Keyboard → game commands. Debug console text→command parsing. Replay serialization. Rebindable keys.                                                                                                                                 | —             |
| 7 | Decorator | <code>StarDecorator</code> , <code>MegaDecorator</code>                                                                                                                                     | Temporary power-up overlays that wrap the active <code>IPlayerState</code> without replacing it.<br><code>StarDecorator(FireState)</code>                                                                                                         | —             |

| #  | Pattern                | Class(es)                                   | Purpose                                                                                                                                                                                                                                                          | Rubric Points |
|----|------------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
|    |                        |                                             | adds invincibility while preserving fireball ability.                                                                                                                                                                                                            |               |
| 8  | <b>Memento</b>         | <code>GameSnapshot</code>                   | Time rewind stores/restores full game state snapshots. Replay system.                                                                                                                                                                                            | —             |
| 9  | <b>Object Pool</b>     | <code>ObjectPool&lt;T&gt;</code>            | Pre-allocated pools for fireballs, particles, Bullet Bills. Avoids allocation in hot loops.                                                                                                                                                                      | —             |
| 10 | <b>Template Method</b> | <code>IMovementStrategy :: execute()</code> | Base strategy <code>execute()</code> defines skeleton:<br><code>calculateTarget()</code> →<br><code>applyMovement()</code> →<br><code>checkConstraints()</code> . Concrete strategies override hooks. Applied to strategies, not to <code>Enemy</code> directly. | —             |

## 4. Class Hierarchy Diagram

---

```

classDiagram
class Entity {
    <<abstract>>
    #sf::Vector2f position
    #sf::Vector2f velocity
    #AABB boundingBox
    #bool active
    +update(float dt)*
    +render(sf::RenderTarget&)*
    +getBoundingBox() AABB
}

class Character {
    <<abstract>>
    #int health
    #float speed
    #float jumpForce
    #bool onGround
    +moveLeft()
    +moveRight()
    +jump()
    +takeDamage()
}

class Player {
    <<abstract>>
    #std::unique_ptr~IPlayerState~ currentState
    #int lives
    #int coins
    #int score
    +powerUp(ItemType)
    +powerDown()
    +shootFireball()
    +getCurrentState() IPlayerState&
}

class Mario {
    +update(float dt)
    +render(sf::RenderTarget&)
}

class Luigi {
    -bool hasDoubleJumped
    +doubleJump()
    +update(float dt)
    +render(sf::RenderTarget&)
}

class Enemy {

```



```

    <<abstract>>
    #std::unique_ptr~IMovementStrategy~ aiStrategy
    #int scoreValue
    +setStrategy(std::unique_ptr~IMovementStrategy~)
    +onStomped()*
    +onHitByFireball()*
}

class Goomba
class KoopaTroopa {
    -bool inShell
    +kickShell()
}

class Item {
    <<abstract>>
    #bool collected
    +activate(Player&)*
    +collect()*
}

class Mushroom
class FireFlower
class Coin
class Star

class Block {
    <<abstract>>
    #bool breakable
    +onHitFromBelow(Player&)*
}

class BrickBlock
class QuestionBlock {
    -Item* containedItem
}

class IPlayerState {
    <<interface>>
    +enter(Player&)*
    +exit(Player&)*
    +handleInput(Player&, InputEvent)*
    +update(Player&, float dt)*
    +getSize() sf::Vector2f*
}

class SmallState
class SuperState
class FireState
class CapeState

```

```

class MiniState

class PlayerStateDecorator {
    <<abstract>>
    #std::unique_ptr~IPlayerState~ wrappedState
    +enter(Player&)
    +exit(Player&)
}
class StarDecorator
class MegaDecorator

Entity <-- Character
Entity <-- Item
Entity <-- Block
Character <-- Player
Character <-- Enemy
Player <-- Mario
Player <-- Luigi
Player *-- IPlayerState
Enemy <-- Goomba
Enemy <-- KoopaTroopa
Item <-- Mushroom
Item <-- FireFlower
Item <-- Coin
Item <-- Star
Block <-- BrickBlock
Block <-- QuestionBlock
IPlayerState <.. SmallState
IPlayerState <.. SuperState
IPlayerState <.. FireState
IPlayerState <.. CapeState
IPlayerState <.. MiniState
IPlayerState <-- PlayerStateDecorator
PlayerStateDecorator <-- StarDecorator
PlayerStateDecorator <-- MegaDecorator

```

## 5. Proposed Folder Tree

---

```

SuperMarioGame/
├── AGENTS.md # AI agent guidelines
├── CS202_FinalProject_SuperMario_Spec.md # Project specification
├── CMakeLists.txt # Build configuration (SFML + ImGui)
├── README.md # Project documentation
├── .gitignore
├── include/ # — HEADER FILES —
│   ├── Core/
│   │   ├── Game.hpp # Singleton game class, owns the main loop
│   │   ├── GameStateManager.hpp # Manages stack of IGameState
│   │   ├── IGameState.hpp # Abstract interface for game states
│   │   ├── MenuState.hpp # Main menu screen
│   │   ├── CharSelectState.hpp # Character selection screen
│   │   ├── PlayingState.hpp # Active gameplay state
│   │   ├── PauseState.hpp # Pause overlay state
│   │   ├── GameOverState.hpp # Game over screen
│   │   ├── VictoryState.hpp # Level/game victory screen
│   │   ├── InputManager.hpp # Keyboard input → Command mapping
│   │   ├── ResourceManager.hpp # Singleton: textures, fonts, sound buffers
│   │   ├── SoundManager.hpp # Singleton: plays SFX and music
│   │   └── EventBus.hpp # Observer pattern: publish/subscribe events
│   ├── Entities/
│   │   ├── Entity.hpp # Abstract base for all game objects
│   │   ├── Character.hpp # Abstract base for players + enemies
│   │   ├── Mario.hpp # Mario-specific logic
│   │   ├── Luigi.hpp # Luigi-specific logic
│   │   ├── Enemy.hpp # Abstract enemy base
│   │   ├── Goomba.hpp # Goomba enemy
│   │   ├── KoopaTroopa.hpp # Koopa Troopa enemy
│   │   ├── Item.hpp # Abstract item base
│   │   ├── Mushroom.hpp # Super Mushroom power-up
│   │   ├── FireFlower.hpp # Fire Flower power-up
│   │   ├── Coin.hpp # Coin collectible
│   │   ├── Star.hpp # Invincibility Star
│   │   ├── Block.hpp # Abstract block base
│   │   ├── BrickBlock.hpp # Breakable brick
│   │   ├── QuestionBlock.hpp # Item-containing ? block
│   │   ├── Pipe.hpp # Warp pipe
│   │   ├── EntityFactory.hpp # Factory pattern: creates entities by type
│   │   ├── IMovementStrategy.hpp # Strategy pattern interface
│   │   ├── PatrolStrategy.hpp # Walk back-and-forth AI
│   │   └── ChaseStrategy.hpp # Chase player AI
│   └── Graphics/
│       ├── Renderer.hpp # SFML rendering wrapper
│       ├── Animation.hpp # Sprite animation (frame-based)
│       └── AnimationManager.hpp # Manages animation definitions

```

|                         |                                                  |
|-------------------------|--------------------------------------------------|
| — Camera.hpp            | # Side-scrolling camera (sf::View)               |
| — HUD.hpp               | # Score, coins, lives, time display              |
| — SpriteSheet.hpp       | # Texture atlas / spritesheet handler            |
| — ParticleSystem.hpp    | # Coin sparkles, brick fragments, etc.           |
| — Physics/              |                                                  |
| — AABB.hpp              | # Axis-Aligned Bounding Box struct               |
| — PhysicsEngine.hpp     | # Gravity, velocity integration                  |
| — CollisionDetector.hpp | # Broad + narrow phase detection                 |
| — CollisionResolver.hpp | # Response: push-out, bounce, damage             |
| — Utils/                |                                                  |
| — Constants.hpp         | # Game-wide constants (gravity, tile size, etc.) |
| — TileMap.hpp           | # Level tilemap loader and renderer              |
| — LevelLoader.hpp       | # Parses level files → TileMap + entities        |
| — Serializer.hpp        | # Save/Load game state to file                   |
| — MathUtils.hpp         | # Vector helpers, clamping, lerp                 |
| — src/                  | # — SOURCE FILES —                               |
| — main.cpp              | # Entry point                                    |
| — Core/                 |                                                  |
| — Game.cpp              |                                                  |
| — GameStateManager.cpp  |                                                  |
| — MenuState.cpp         |                                                  |
| — CharSelectState.cpp   |                                                  |
| — PlayingState.cpp      |                                                  |
| — PauseState.cpp        |                                                  |
| — GameOverState.cpp     |                                                  |
| — VictoryState.cpp      |                                                  |
| — InputManager.cpp      |                                                  |
| — ResourceManager.cpp   |                                                  |
| — SoundManager.cpp      |                                                  |
| — EventBus.cpp          |                                                  |
| — Entities/             |                                                  |
| — Entity.cpp            |                                                  |
| — Character.cpp         |                                                  |
| — Mario.cpp             |                                                  |
| — Luigi.cpp             |                                                  |
| — Enemy.cpp             |                                                  |
| — Goomba.cpp            |                                                  |
| — KoopaTroopa.cpp       |                                                  |
| — Item.cpp              |                                                  |
| — Mushroom.cpp          |                                                  |
| — FireFlower.cpp        |                                                  |
| — Coin.cpp              |                                                  |
| — Star.cpp              |                                                  |
| — Block.cpp             |                                                  |
| — BrickBlock.cpp        |                                                  |
| — QuestionBlock.cpp     |                                                  |
| — Pipe.cpp              |                                                  |

```

├── EntityFactory.cpp
├── PatrolStrategy.cpp
├── ChaseStrategy.cpp
├── Graphics/
│   ├── Renderer.cpp
│   ├── Animation.cpp
│   ├── AnimationManager.cpp
│   ├── Camera.cpp
│   ├── HUD.cpp
│   ├── SpriteSheet.cpp
│   └── ParticleSystem.cpp
├── Physics/
│   ├── AABB.cpp
│   ├── PhysicsEngine.cpp
│   ├── CollisionDetector.cpp
│   └── CollisionResolver.cpp
├── Utils/
│   ├── Constants.cpp
│   ├── TileMap.cpp
│   ├── LevelLoader.cpp
│   ├── Serializer.cpp
│   └── MathUtils.cpp
├── assets/
│   ├── textures/
│   │   ├── mario_spritesheet.png
│   │   ├── enemies_spritesheet.png
│   │   ├── items_spritesheet.png
│   │   ├── tileset.png
│   │   └── backgrounds/
│   ├── sounds/
│   │   ├── sfx/
│   │   └── music/
│   ├── fonts/
│   │   └── PressStart2P.ttf
│   └── levels/
│       ├── level_1.json
│       ├── level_2.json
│       └── level_3.json
├── saves/
│   └── .gitkeep
├── logs/
│   └── agent_history.log

```

# Sprite sheets, tilesets, backgrounds

# Jump, coin, stomp, power-up, death

# Level BGM, menu BGM, game over

# Retro pixel font

# Level data files

# Saved game states

# Agent interaction log

## 6. Spec Questionnaire — Please Fill In

---

Below is the questionnaire form. Please fill in your answers or preferences for each question. I will use your answers to create the final detailed specification and task breakdown.

---

### A. Architecture & Tech Stack

| #  | Question                                                                                                                                 | Your Answer  |
|----|------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| A1 | <b>C++ Standard:</b> Target C++17 or C++20?                                                                                              | C++17        |
| A2 | <b>Confirm Engine:</b> SFML 3.0.2 (recommended, already set up). Accept?                                                                 | Accept       |
| A3 | <b>ImGui:</b> Keep ImGui-SFML for dev tools / level editor UI?                                                                           | Accept       |
| A4 | <b>Save/Load Format:</b> JSON (human-readable) or binary (compact)?                                                                      | JSON         |
| A5 | <b>Save Data Scope:</b> What must be persisted? (e.g., current level, score, lives, coins, player form, time remaining, exact position?) | All of those |

---

## B. Gameplay Mechanics & Physics

| #  | Question                                                                                                                                                                        | Your Answer                                                                          |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| B1 | <b>Tile Size:</b> 16×16 pixels (classic NES) or 32×32 (scaled)?                                                                                                                 | 32×32                                                                                |
| B2 | <b>Window Resolution:</b> 1024×768, 1280×720, or other?                                                                                                                         | 1280×720                                                                             |
| B3 | <b>Physics Style:</b> Custom AABB physics (our implementation) or integrate Box2D as a physics engine?                                                                          | Custom AABB                                                                          |
| B4 | <b>Fixed Timestep:</b> Use a fixed physics timestep (e.g., 1/60s) with interpolated rendering?                                                                                  | Yes                                                                                  |
| B5 | <b>Gravity Value:</b> Use NES-authentic $\sim 0.4 \text{ px/frame}^2$ or custom? If custom, specify.                                                                            | Custom, expose this to ImGui, start with initial value: $0.5 \text{ px/frame}^2$     |
| B6 | <b>Mario Speed:</b> Max walking speed and max running speed (pixels/sec)? Leave blank for NES-authentic defaults.                                                               | Custom, expose this to ImGui, start with initial value, of about 150px/s and 300px/s |
| B7 | <b>Jump Height:</b> How high should Mario jump in tiles? (NES Mario jumps $\sim 4$ tiles high)                                                                                  | Custom, expose this to ImGui, start with initial value, of about 4 tiles high        |
| B8 | <b>Luigi Differences:</b> Exact multipliers? (Default proposal: jump_force $\times 1.2$ , speed $\times 0.85$ , slightly floatier = lower gravity while airborne $\times 0.9$ ) | Accept, maybe introduce double jump                                                  |
| B9 | <b>Fireball Mechanics:</b> How far do fireballs travel? Bounce on ground? Max active at once?                                                                                   | Bounces on the ground, destroyed on wall impact. Max 2 active at once                |



## C. Characters & Enemies

| #  | Question                                                                                                                             | Your Answer                                                                                                                     |
|----|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| C1 | <b>Player Characters:</b> Mario + Luigi only, or add more?                                                                           | Just those currently                                                                                                            |
| C2 | <b>Character Switching:</b> Switch during gameplay (hotkey) or only at character select screen?                                      | Both                                                                                                                            |
| C3 | <b>Player States:</b> Small → Super (Mushroom) → Fire (Fire Flower). Add Star (invincible, timed)? Others?                           | Small, Super, Fire, and Star (temporary timer overlay)                                                                          |
| C4 | <b>Enemy Roster (MVP):</b> Which enemies for the first build? Proposal: Goomba, Koopa Troopa, Koopa Paratroopa (flying). Add/remove? | Accept                                                                                                                          |
| C5 | <b>Goomba AI:</b> Walk in one direction, reverse on wall collision, fall off ledges?                                                 | Accept                                                                                                                          |
| C6 | <b>Koopa AI:</b> Same as Goomba but retreats into shell when stomped? Can shell be kicked?                                           | Accept                                                                                                                          |
| C7 | <b>Proximity AI:</b> Should any enemy actively chase Mario when within a certain pixel range? If so, which enemies and what range?   | Create a "Boo" (Ghost) enemy or a Swooping Paratroopa that stays idle until Mario is within ~250 pixels, then moves toward him. |
| C8 | <b>Boss Enemies:</b> Include a boss at end of Level 3? (e.g., Bowser-like character)                                                 | Accept                                                                                                                          |

## D. Items & Power-ups

| #  | Question                                                                                                   | Your Answer                          |
|----|------------------------------------------------------------------------------------------------------------|--------------------------------------|
| D1 | <b>Item Roster (MVP):</b> Super Mushroom, Fire Flower, Coin, Star. Add 1-UP Mushroom? Others?              | Accept                               |
| D2 | <b>Coin Goal:</b> Collecting 100 coins = 1 extra life?                                                     | Yes                                  |
| D3 | <b>Star Duration:</b> How many seconds does invincibility last? (NES: ~10 seconds)                         | Yes                                  |
| D4 | <b>Item Spawning:</b> Items pop out of QuestionBlocks only, or also from hidden blocks / brick blocks?     | Items pop out of QuestionBlocks only |
| D5 | <b>Power-down Rule:</b> Fire Mario hit → becomes Small Mario directly, or steps down to Super Mario first? | Fire → Super → Small                 |

---

## E. Levels & World Design

| #  | Question                                                                                                                                                                                                 | Your Answer                                                                          |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| E1 | <b>Number of Levels:</b> 3 (as required). Each level = 1 continuous stage, or sub-stages?                                                                                                                | 3 continuous stages with checkpoints                                                 |
| E2 | <b>Level Themes:</b> Proposal — Level 1: Overworld/Grassland, Level 2: Underground/Cave, Level 3: Castle/Lava. Accept or change?                                                                         | Overworld, Underground, Castle                                                       |
| E3 | <b>Level Width:</b> How wide should levels be? (NES World 1-1 ≈ 210 tiles wide, ~3360px at 16px/tile). Approximate length?                                                                               | 200 tiles wide                                                                       |
| E4 | <b>Level Height:</b> Visible screen height in tiles? (NES = 15 tiles of 16px = 240px visible, but our window is bigger)                                                                                  | 22.5 tiles                                                                           |
| E5 | <b>Difficulty Progression:</b> What makes each level harder? Proposal — L1: few enemies, flat terrain; L2: gaps/pits, more enemies, moving platforms; L3: lava pits, high enemy density, timed sections. | Accept                                                                               |
| E6 | <b>Level End:</b> Flagpole (classic Mario) or reaching a specific door/point?                                                                                                                            | Flagpole                                                                             |
| E7 | <b>Warp Pipes:</b> Include pipes that teleport the player to bonus areas or between sections?                                                                                                            | Yes, include warp pipes that teleport the player to bonus areas or between sections. |
| E8 | <b>Level File Format:</b> Store levels as JSON (easy to edit/debug), TMX (Tiled editor), or custom text format?                                                                                          | JSON                                                                                 |
| E9 | <b>Level Editor (Bonus):</b> Include in scope for MVP or defer to post-MVP? If included, ImGui-based or standalone?                                                                                      | Defer to Bonus/Post-MVP                                                              |

## F. UI, Screens & Game Flow

| #  | Question                                                                                                                                                      | Your Answer                                                                      |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| F1 | <b>Screen Flow:</b> Proposal —<br>Main Menu → Character Select → Level X → (Pause) → Level Complete → (next level or Victory) → Game Over . Accept or modify? | Accept                                                                           |
| F2 | <b>HUD Elements:</b> Score, Coins (xcounter), World/Level name, Time countdown, Lives remaining. Add/remove?                                                  | Accept                                                                           |
| F3 | <b>Time Limit:</b> Should each level have a countdown timer? (NES: 300–400 game seconds). How long?                                                           | 300 game seconds                                                                 |
| F4 | <b>Lives System:</b> Start with 3 lives? Game Over when lives = 0? Continue from checkpoint or restart level?                                                 | Start with 3. Game Over = return to Main Menu, if die, respawn at the checkpoint |
| F5 | <b>Pause Menu Options:</b> Resume, Restart Level, Return to Main Menu, Quit?                                                                                  | Accept                                                                           |
| F6 | <b>Main Menu Options:</b> New Game, Load Game, Options (volume/controls), Quit?                                                                               | Accept                                                                           |
| F7 | <b>Score Display:</b> Show high scores on main menu? Persist high scores across sessions?                                                                     | Accept                                                                           |

## G. Graphics & Assets

| #  | Question                                                                                                | Your Answer     |
|----|---------------------------------------------------------------------------------------------------------|-----------------|
| G1 | <b>Art Style:</b> NES pixel-art (retro 8-bit), or more modern/polished sprites (16-bit SNES style)?     | 16-bit SNES     |
| G2 | <b>Sprite Source:</b> Use open-source Mario-style sprites (e.g., from OpenGameArt), or create custom?   | Open source     |
| G3 | <b>Sprite Dimensions:</b> Base tile/sprite size in pixels? (16×16, 32×32, or other)                     | 32×32           |
| G4 | <b>Scrolling:</b> Horizontal only (classic Mario), or also vertical scrolling in some sections?         | Horizontal only |
| G5 | <b>Background Layers:</b> Parallax scrolling with multiple background layers (clouds, mountains, etc.)? | Accept          |
| G6 | <b>Particle Effects:</b> Brick breaking particles, coin sparkles, death poof animations?                | Accept          |
| G7 | <b>Screen Transitions:</b> Fade-in/fade-out between levels, or instant cut?                             | Accept          |

---

## H. Audio

| #  | Question                                                                                                                 | Your Answer |
|----|--------------------------------------------------------------------------------------------------------------------------|-------------|
| H1 | <b>Sound Effects Needed:</b> Jump, coin, stomp, power-up, power-down, fireball, 1-UP, death, flagpole, pipe. Add/remove? | Accept      |
| H2 | <b>Background Music:</b> Separate BGM per level theme? Menu BGM? Game Over jingle?                                       | Accept      |
| H3 | <b>Audio Source:</b> Use royalty-free retro-style sounds, or source from open game audio libraries?                      | Open source |
| H4 | <b>Volume Controls:</b> In-game volume slider for SFX and Music separately?                                              | Accept      |

---

## I. Advanced & Bonus Scope

| #  | Question                                                                                                                                              | Your Answer                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| I1 | <b>3D Game (5 bonus pts):</b> Is the team targeting the 3D bonus points, or staying strictly 2D? If 3D, what approach? (e.g., 2.5D with depth layers) | 2D                                                      |
| I2 | <b>Multiplayer:</b> Two players on same keyboard (co-op or versus), or single-player with character select only?                                      | Two players on same keyboard, versus                    |
| I3 | <b>Level Editor Priority:</b> Must-have for submission, or nice-to-have if time allows?                                                               | Nice-to-have                                            |
| I4 | <b>Controller Support:</b> Keyboard only, or also gamepad/controller support?                                                                         | Keyboard only, abstraction for later gamepad/controller |

---

## 7. Proposed Development Phases

Once specs are finalized, the implementation will be organized into phases on feature branches off `dev` :

| Phase    | Branch                 | Deliverables                                                                       | Est. Duration   |
|----------|------------------------|------------------------------------------------------------------------------------|-----------------|
| Phase 0  | dev                    | ✓ Done — Environment setup, SFML + ImGui compiled                                  | Complete        |
| Phase 1  | feature/core-engine    | Game loop, GameStateManager, InputManager, ResourceManager, SoundManager, EventBus | 1 week          |
| Phase 2  | feature/physics        | AABB, PhysicsEngine, CollisionDetector, CollisionResolver, gravity/velocity        | 1 week          |
| Phase 3  | feature/entities       | Entity hierarchy, Mario, Luigi, Goomba, KoopaTroopa, Items, Blocks, EntityFactory  | 1–2 weeks       |
| Phase 4  | feature/tilemap-levels | TileMap, LevelLoader, Camera scrolling, 3 level files                              | 1 week          |
| Phase 5  | feature/graphics       | Animation system, SpriteSheet, HUD, ParticleSystem, asset integration              | 1 week          |
| Phase 6  | feature/audio          | SoundManager wiring, SFX + BGM loading, volume controls                            | 3 days          |
| Phase 7  | feature/game-states    | Menu, CharSelect, Playing, Pause, GameOver, Victory screens                        | 1 week          |
| Phase 8  | feature/save-load      | Serializer (JSON), save/load game progress                                         | 3 days          |
| Phase 9  | feature/enemy-ai       | Strategy pattern AI, proximity chase, difficulty tuning                            | 1 week          |
| Phase 10 | feature/polish         | Bug fixes, balancing, transitions, particles, edge cases                           | 1 week          |
| Bonus    | feature/level-editor   | ImGui-based level editor                                                           | If time permits |

**[!IMPORTANT] Please fill in Section 6 (the questionnaire) above.** Your answers will determine the final detailed specification, physics constants, asset pipeline, and task granularity for each development phase. I will not proceed with implementation until specs are locked down.